COMP1005 – Fundamentals of Programming

Project Report for Disease Simulation

Contents

1. Overview
2. User Guide
3. Traceability Matrix
4. Discussion
5. Showcase
6. Conclusion
7. Future Work
8. References

Overview

This report presents a disease simulation project built in Python. The program models the spread of a disease across a grid representing a population, incorporating features such as infection, recovery, death, and movement barriers. The simulation also includes parameters for population size, infection probability, and recovery probability.

The main features of the program include:

- A grid-based representation of the population with adjustable size.
- Disease states including infected, uninfected, recovered, and deceased.
- Visualization of the simulation process using the matplotlib library.
- Barriers and airports to influence movement dynamics.
- Support for different neighborhood configurations (Moore or Von Neumann).

The simulation files include:

1. **DiseaseSim.py** – Implements the simulation's logic, visualization, and user input handling.
2. **Libraries** – Utilizes Python's numpy, matplotlib.pyplot, and random libraries for computation and visualization.

User Guide

Main Requirements

Before running the simulation, ensure Python 3.x and the necessary libraries (numpy and matplotlib) are installed. An IDE such as Spyder or Jupyter Notebook is recommended for ease of use.

Running the Simulation

1. Open the code file in your IDE.
2. Run the script. The program will prompt for the following inputs:
    - Grid size (rows and columns).
    - Number of simulation steps.
    - Initial population size.
    - Initial infected population.
    - Neighborhood type (moore or von Neumann).

3. The program will display a visualization of the simulation at each step and output the final statistics.

Traceability Matrix

Feature	Code Reference	Test Reference	Completion Date
Population Distribution	distribute()	_	N/A
Neighborhood Calculation and Selection	get_neighbourhood_type(), get_neighbourhood	Verified proper error handling for invalid inputs using if-else blocks.	N/A
Movement Dynamics	movePeeps()	_	N/A
Infection Spread	infect()	_	N/A
Recovery and Death Dynamics	recover_and_die()	_	N/A
Airport and Barrier setup	setup_airports_and_barriers()	_	N/A
Grid Visualization	plotGrids()	_	N/A
Input Validation for Simulation	get_simulation_parameters()	Tested invalid input handling for negative values, non-integers, and logical errors using try-except.	N/A
Simulation Execution	simulate(), main script	_	N/A
State Display	displayGrid()	_	N/A
Final Statistics Output	print_final_stats()	_	N/A

Scatter Plot Creation	makeScatter()	_	N/A

Discussion of Implemented Features

The implemented features for the disease simulation are outlined below, with references to the Traceability Matrix. Each feature is described in terms of its functionality and implementation approach:

1. Neighborhood Calculation

- **Code Reference**: get_neighbours(r, c, num_r, num_c, neighbourhood_type="moore")
- **Purpose**: To determine the neighboring cells for a given cell (r, c) based on the chosen neighborhood type (Moore or von Neumann).

- **Implementation**:

- It accepts the row (r) and column (c) indices of the current cell, the total number of rows (num_r) and columns (num_c) in the grid, and the neighbourhood_type (default is Moore).
- In Moore's neighborhood, it checks all eight surrounding cells (up, down, left, right, and diagonals).
- In von Neumann's neighborhood, it only checks the four orthogonal neighbors (up, down, left, right).
- The valid neighbors are returned as a list of tuples containing their row and column indices.

2. Neighborhood Selection

- **Code Reference:** get_neighbourhood_type()
- **Purpose**: To prompt the user for the type of neighborhood(Moore or Von neumann)

- **Implementation:**

- It asks the user to input the desired neighborhood type.
- If the input is valid (moore or von Neumann), it returns the input. Otherwise, it asks the user again.

3. Population Distribution

- **Code Reference**: distribute(grid, num_r, num_c, numpeep)
- **Purpose**: To randomly distribute numpeep individuals across the grid.

- **Implementation**:

- It iterates numpeep times to place one individual randomly in a grid cell by selecting random row (rpos) and column (cpos) indices.

- It increases the count at grid[rpos, cpos] by 1 for each individual placed.
- This function is used for placing both infected and uninfected individuals in the grid at the start of the simulation.

4. Movement Dynamics

- **Code Reference**: movePeeps(cur, next, r, c, NUM_ROWS, NUM_COLS, barriers)
- **Purpose:** To move individuals from the current grid to a new position based on random movement, avoiding barriers and grid boundaries.

- **Implementation:**
- It iterates over each individual in the cell (r, c) of the cur grid.
- Each individual moves randomly by one step (up, down, left, right) within the grid boundaries.
- If the movement leads to a valid (non-barrier, within-bound) position, the individual is moved to that new cell in the next grid.
- If the movement is invalid (either out of bounds or into a barrier), the individual remains in the current cell.
- This function is called for both infected and uninfected individuals to update their positions in the simulation.

5. Infection Spread

- **Code Reference**: infect(inf, notinf, r, c, prob)
- **Purpose:** To simulate the infection process in a grid cell, spreading the infection from infected individuals to uninfected ones.

- **Implementation:**

  - It scales the infection probability (prob) by the number of infected individuals in the cell (r, c).
  - Then, for each uninfected individual in that cell, it generates a random value and compares it to the scaled infection probability.
  - If the random value is less than the probability, the individual becomes infected, and the count of infected and uninfected individuals is updated accordingly.

6. Recovery and Death Dynamics

- **Code Reference**: recover_and_die(infected, recovered, deceased, immunity, death_prob, recovery_prob, r, c)
- **Purpose:** To simulate the recovery or death of infected individuals in a grid cell.
- **Implementation:**
- It iterates over each infected individual in the cell (r, c).
- For each infected individual, it generates a random number to determine if they die (death_prob) or recover (recovery_prob).
- If the individual dies, the count of deceased individuals is increased, and the infected count is decreased.

- If the individual recovers, the count of recovered individuals is increased, the infected count is decreased, and the immunity of the cell is set to `1` (indicating immunity).

7. Airport and Barrier Setup

- **Code Reference**: setup_airports_and_barriers(NUM_ROWS, NUM_COLS, num_airports, num_barriers)
- **Purpose:** To place airports and barriers randomly on the grid.
- **Implementation:**
- The function randomly places num_airports and num_barriers on the grid by generating random row and column coordinates for each.
- It returns two lists:
  - airports: a list of coordinates where airports are located.
  - barriers: a list of coordinates where barriers are located.

- Airports and barriers are later used for visualizing the simulation and restricting movement.

8. Grid Visualization

- **Code Reference**: plotGrids(infected, uninfected, recovered, deceased, airports, barriers, NUM_ROWS, NUM_COLS, timestep)
- **Purpose:** To visualize the grid state at each timestep of the simulation.
- **Implementation:**
- It creates a scatter plot of the simulation, using different colors to represent the state of individuals (infected, uninfected, recovered, deceased).
- The status_map dictionary stores the color coding for each grid cell. Initially, it sets all cells to `'b'` (blue for uninfected).
- It then plots the individuals' states with different colors:
  - **Red (`r`)** for infected.
  - **Blue (`b`)** for uninfected.
  - **Green (`g`)** for recovered.
  - **Black (`k`)** for deceased.
- Airports and barriers are also plotted with distinct markers and colors.
- The plot is displayed with a legend, a title indicating the current timestep, and a visual representation of the grid state.

9. Input Validation for Simulation

- **Code Reference**: get_simulation_parameters()
- **Purpose:** To prompt the user for the simulation parameters such as grid size, number of steps, and initial populations.
- **Implementation:**
- It takes the user input for the number of rows, columns, simulation steps, initial population, and initial infected population.

- The input is validated to ensure all values are positive integers and that the number of infected individuals does not exceed the total population.
- If the inputs are valid, it returns the parameters. Otherwise, it prints an error message and returns None.

10. State Display

- **Code Reference**: displayGrid(grid, num_r, num_c)
- **Purpose:** To display the grid in a human-readable format.

- **Implementation:**
- It loops through each row and column of the grid and prints the value of each cell.
- The cells in a row are printed in one line, and after each row, a new line is printed to separate rows visually.

11. Final Statistics Output

- **Code Reference**: print_final_stats(infected, uninfected, recovered, deceased, NUM_ROWS, NUM_COLS)
- **Purpose:** To print the final statistics of the simulation after all timesteps have been completed.

- **Implementation:**
- It calculates the total number of infected, uninfected, recovered, and deceased individuals by summing the respective grids.
- It prints the final statistics and the total population, including the size of the grid.

12. Simulation Execution

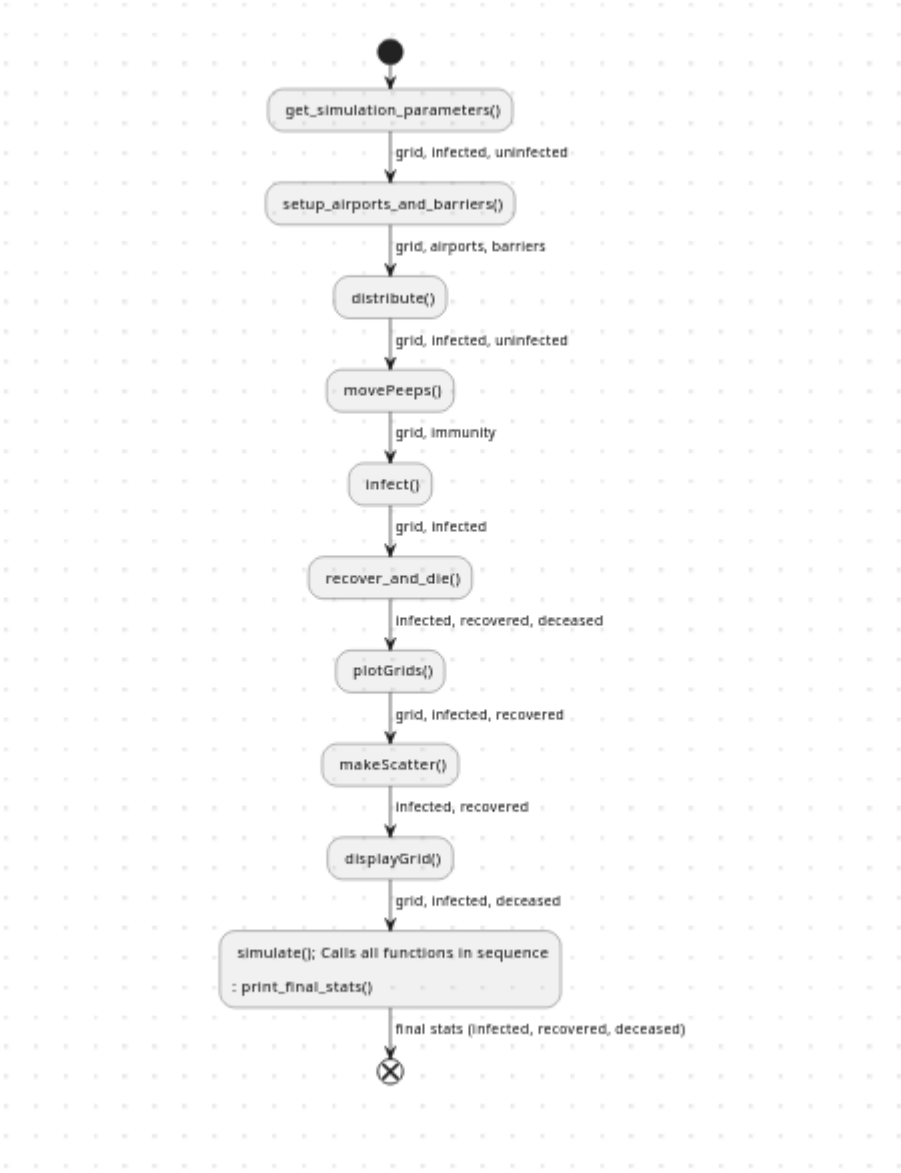
- **Code Reference**: simulate(NUM_ROWS, NUM_COLS, NUM_STEPS, INIT_POP, INIT_INFECTED, neighbourhood_type="moore")
- **purpose:** This is the main simulation function that runs the entire disease simulation.

- **Implementation:**
- It initializes the infected, uninfected, recovered, deceased, and immunity grids.
    - It distributes the initial population of infected and uninfected individuals using the distribute function.
    - It sets up the airports and barriers.
    - It then loops through each timestep, performing the infection, movement, and recovery/death processes.
    - After each timestep, it visualizes the grid state using plotGrids and updates the grid for the next timestep.
    - At the end of the simulation, it prints the final statistics using print_final_stats.

Main Program:

The main part of the code collects the simulation parameters from the user and then starts the simulation by calling simulate function.

UML DIAGRAM


get_simulation_parameters()
grid, infected, uninfected
setup_airports_and_barriers()
grid, airports, barriers
distribute()
grid, infected, uninfected
movePeeps()
grid, immunity
infect()
grid, infected
recover_and_die()
infected, recovered, deceased
plotGrids()
grid, infected, recovered
makeScatter()
infected, recovered
displayGrid()
grid, infected, deceased
simulate(); Calls all functions in sequence
: print_final_stats()
final stats (infected, recovered, deceased)

Showcase

Introduction

For this showcase, I have chosen to set up three different scenarios for the disease simulation code, each with varying grid sizes, population densities, and initial infected individuals. These scenarios allow us to observe the effects of these factors on the spread of a disease over multiple time steps.

Simulation Setup:

- **Input Files**: There are no input files required for this simulation as it is generated entirely within the program based on parameters passed to the simulate function.

Parameters:

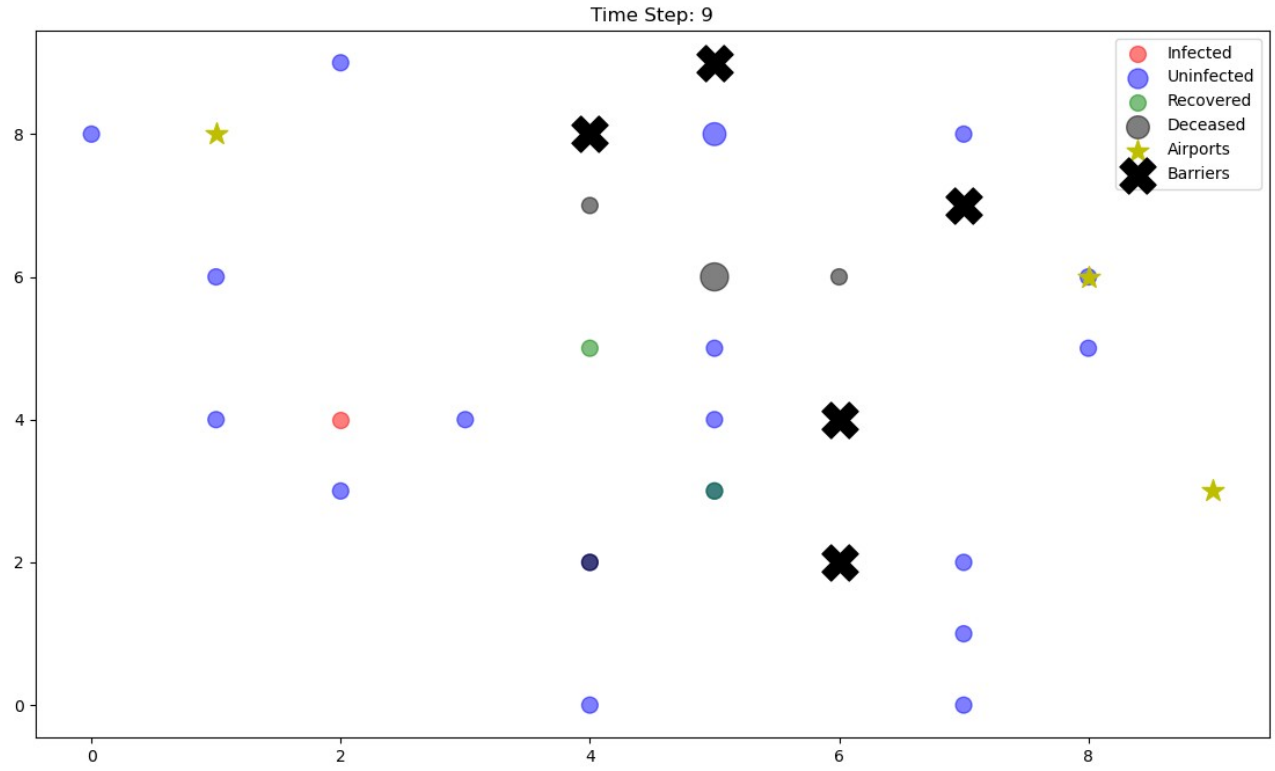
- NUM_ROWS and NUM_COLS: These define the grid size (i.e., number of rows and columns).
- NUM_STEPS: This is the number of time steps the simulation will run.
- INIT_POP: The initial population in the simulation.
- INIT_INFECTED: The number of initially infected individuals.
- neighbourhood_type: Determines the neighbourhood model used for disease spread (either "moore" for 8 neighbours or "von Neumann" for 4 neighbours).

Scenario 1: Small Grid with Low Population Density

Input:

- Grid: 10x10
- Steps: 10
- Initial Population: 20
- Initial Infected: 1
- Neighbourhood type: moore(8 neigbours)

Outcome: In this scenario, the disease has spread slowly due to the low initial number of infected individuals and the small population. The movement between individuals is minimal, and barriers and airports will have little to no effect due to the small grid size. Most of the individuals are uninfected, only one is infected while a few have recovered as you can observe in the picture below.



Total Infected: 2

Total Uninfected: 19

Total Recovered: 2

Total Deceased: 6

Grid Size: 10 rows by 10 columns

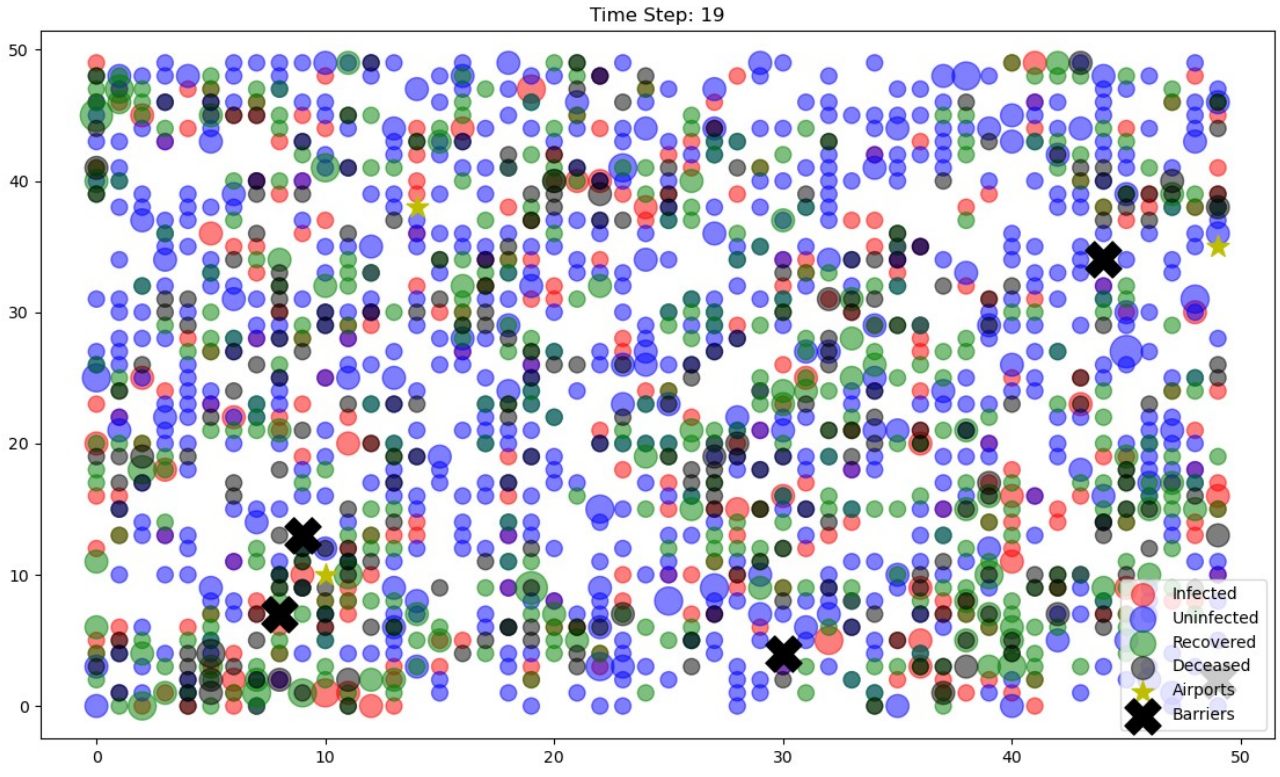
Total Population: 29

Scenario 2: Large Grid with High Population Density and High Infected Population

Input:

- Grid: 50x50
- Steps: 20
- Initial Population: 1000
- Initial Infected: 200
- Neighborhood type: von neumann (4 neighbors)

Outcome: In this scenario, the disease has spread rapidly as compared to previous scenario due to the high number of initially infected individuals and the large population. The grid size allows for more complex interactions between individuals. The number of airports and barriers has impacted the movement but has not halted the spread entirely.



Total Infected: 382

Total Uninfected: 818

Total Recovered: 526

Total Deceased: 302

Grid Size: 50 rows by 50 columns

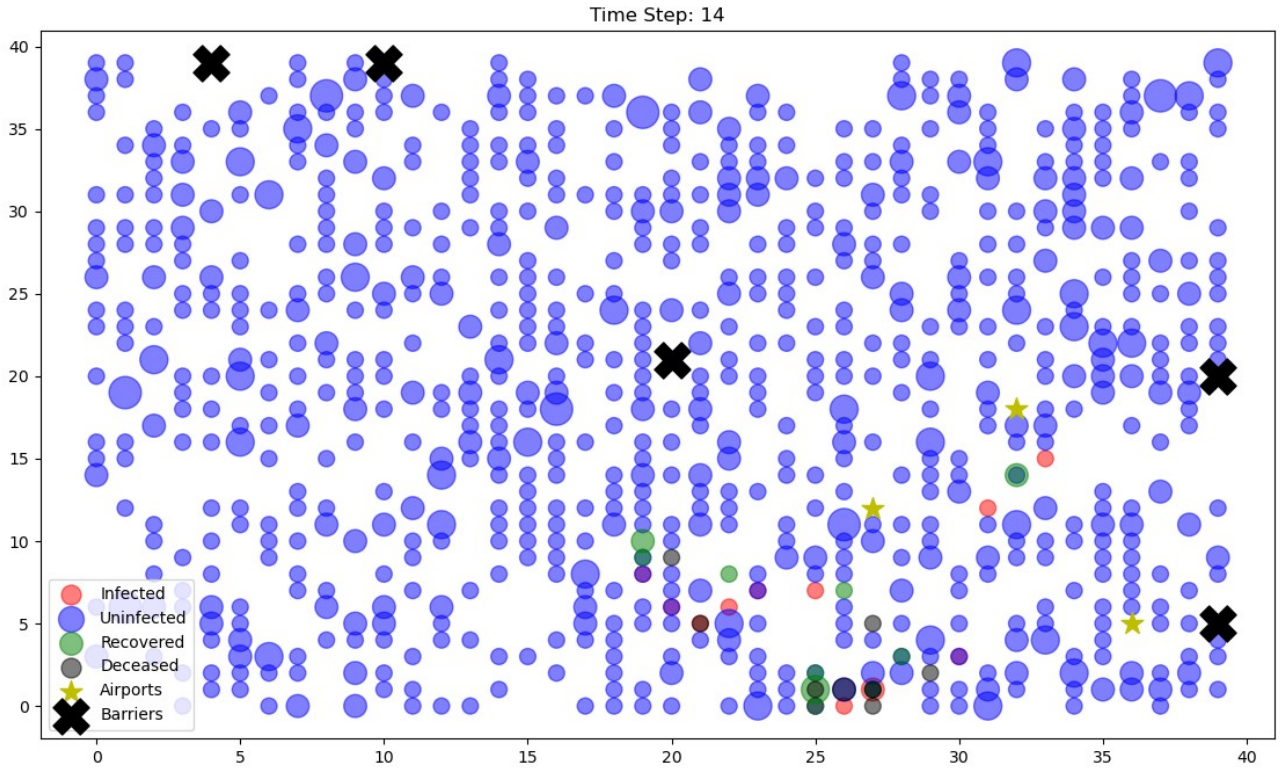
Total Population: 2028

Scenario 3: Large Grid with Low Initial Infection Rate

Input:

- Grid: 40x40
- Steps: 15
- Initial Population: 1000
- Initial Infected: 5
- Neighborhood type: moore(8 neighbors)

Output: In this scenario, the disease has not spread rapidly because of less initial infected people, as you can see in the picture below. You will see most of the people are uninfected, with only few infected, and some recoveries as well.



Total Infected: 16

Total Uninfected: 989

Total Recovered: 14

Total Deceased: 9

Grid Size: 40 rows by 40 columns

Total Population: 1028

Conclusion

In this project, we successfully simulated the spread of a disease across a grid-based population, taking into account various factors such as infection probability, recovery, death, and movement constraints (via barriers and airports). The model allowed for the exploration of different neighbourhood types (Moore and Von Neumann) to observe how these changes affect the spread of the disease.

The key features of the simulation included:

- **Population Initialization**: A grid where individuals (infected or uninfected) are randomly distributed.
- **Infection Spread**: A probabilistic model to simulate the transmission of the disease from infected individuals to their uninfected neighbours.
- **Movement Constraints**: The simulation incorporated barriers that restricted movement and airports that acted as accelerants to disease spread.
- **Visualization**: The use of matplotlib for dynamic and clear visual representations of the population’s state at each timestep.

The project demonstrated how simulations could offer valuable insights into disease transmission dynamics. By adjusting parameters such as population density, infection rates, and movement limitations, we can simulate and explore how different factors contribute to the progression or containment of the disease.

In summary, this simulation helped visualize and understand key aspects of disease outbreaks, showcasing the potential for such models in public health planning and epidemic forecasting.

In my code, i have chosen a procedural programming approach, which focuses on using functions to perform specific tasks in a linear flow. The decision to avoid using classes is based on the simplicity of the simulation, where data (like grid, population) is passed directly between functions without the need to encapsulate them in objects.

Classes are typically useful when managing complex entities with shared behaviors and attributes, but in this case, the simulation's straightforward structure doesn't require them.

However, if the simulation becomes more complex in the future, refactoring to include classes could improve organization and scalability.

Future Work

While this disease simulation project provides a basic framework for understanding disease spread, there are several areas for improvement and further investigation. Some possible extensions and enhancements include:

1. Incorporating Variable Infection Rates:

- Currently, the infection rate is constant. Future work could involve implementing varying infection rates depending on factors such as the proximity of infected individuals, the age or health condition of the population, or the presence of vaccination.

2. Visualization Enhancements:

- Although the current visualization shows the status of individuals at each timestep, future work could improve the presentation. For example, using heatmaps to represent areas with high infection rates or animating the progression of the disease across the grid could provide better insights into the dynamics of the outbreak.

3. Command-Line Arguments and Parameter Sweep:

- **Command-line arguments**: The simulation could be enhanced by using command-line arguments to automate the input of parameters like grid size, number of steps, initial population, infection rate, etc. This approach makes the program more flexible and user-friendly, especially when integrating it into larger simulation pipelines. For instance, users could specify simulation parameters directly in the terminal or script without needing to interact with a prompt.
- **Parameter Sweep**: With the use of command-line arguments, it would be beneficial to implement a parameter sweep feature. This feature would allow the simulation to automatically iterate through various combinations of parameters (e.g., population density, infection rates, recovery rates) and run multiple simulations.
    - ***Benefits*:** A parameter sweep helps in understanding how different parameters affect the disease spread over time. This is particularly useful for sensitivity analysis, where researchers want to identify the key factors influencing the outbreak's behavior.
- Implementing this would automate large-scale simulations, saving time and allowing for quicker insights into how different factors affect the disease dynamics.

References

1. Python Software Foundation. (2023). *random — Generate pseudo-random numbers*.

2. NumPy. (2020). *NumPy: The fundamental package for scientific computing in Python*.
3. Matplotlib. (2007). *Matplotlib: A 2D graphics library*.
4. Anaconda, Inc. (2023). *Free Download | Anaconda.* https://www.anaconda.com/download.
5. Curtin University. (2024). *COMP1005 – Fundamentals of Programming Prac 01-10.*